Google Performance Tools

- 1. 简介
- 2. 安装与配置
  - 2.1 thread-caching malloc
  - 2.2 heap-profiler using tcmalloc
  - 2.3 heap-checker using tcmalloc
  - 2.4 cpu-profiler
  - 2.5 动态链接
  - 2.6 pprof
- 3 应用
  - 3.1 thread-caching
  - 3.2 heap-profiler
  - 3.3 heap-checker
  - 3.4 cpu-profiler
- 4 参考文献

1. 简介

`google-perftools` 是google开发的一款非常实用的工具集，主要包括：

- 性能优异的 `malloc free` 内存分配器 tcmalloc
- 基于 tcmalloc 的堆内存检测工具 heap-profiler
- 基于 tcmalloc 的堆内存泄漏分析工具 heap-checker
- 基于 tcmalloc 实现的程序 CPU 性能监测工具 cpu-profiler

2. 安装与配置

- 使用 `apt` 命令安装

  ```
  sudo apt install google-perftools
  sudo apt install libprotobuf-dev
  sudo apt install libtcmalloc-minimal4
  sudo apt install libgoogle-perftools-dev
  sudo apt install libunwind-dev
  ```

- 通过源码安装 gperftools
  1. https://github.com/gperftools/gperftools

2. 进入源码根目录

```
./autogen.sh       # 生成配置文件
./configure        # 生成makefile
make               # build
sudo make install  # install
# 相关依赖工具包
sudo apt install graphviz
sudo apt install gv
sudo apt install kcachegrind
/usr/local/lib     # 默认安装lib文件所在目录
/usr/local/include # 默认安装头文件所在目录
```

- 通过源码安装 libunwind

```
# 获取源码方式一
git clone https://github.com/libunwind/libunwind
# 获取源码方式二
wget http://download.savannah.gnu.org/releases/libunwind/libunwind-0.99-beta.tar.gz
cd libunwind
./autogen.sh       # 生成配置文件
./configure        # 生成makefile
make               # build
sudo make install  # install
/usr/local/lib     # 默认安装lib文件所在目录
/usr/local/include # 默认安装头文件所在目录
```

> 在64位操作系统下需要 libunwind 支持。

2.1 thread-caching malloc

在链接的时候增加参数 `-ltcmalloc` 或 `-ltcmalloc_minimal` 替代代码中的 `malloc` 和 `new` ，从而使用 tcmalloc。

tcmalloc 功能可在我们测试过的所有系统上使用，参考源码根目录的 INSTALL 文件。参考源码根目录的 README_windows.txt 文件了解 windows 上面 tcmalloc 的用法。

在编译的时候gcc会做一些优化，假设它在用自己内置的malloc，显然这个假设对于 tcmalloc 来说是错误的。虽然在实践中，这种做法我们目前还没有发现什么问题，但是对于用了 `-ltcmalloc` 参数的用户来说，出问题的风险最高（using gperftools/malloc_hook.h），对于使用 `-tcmalloc_minimal` 参数风险相对低一些。

2.2 heap-profiler using tcmalloc

快速上手：

1. 链接时加上参数 `-ltcmalloc`

   ```
   gcc [...] -o xxx -ltcmalloc
   ```

2. 设置 `HEAPPROFILE` 环境变量：

   ```
   HEAPPROFILE=/tmp/heapprof <path/to/binary> [binary args]
   ```

3. 使用 pprof 来分析 heap 的使用率

   ```
   pprof <path/to/binary> /tmp/heapprof       # run 'ls' to see options
   pprof --gv <path/to/binary> /tmp/heapprof
   ```

2.3 heap-checker using tcmalloc

为了捕获所有堆泄漏，必须将 tcmalloc 链接到您的可执行文件中。heap-checker 可能在链接行后列出的库中对某些内存访问进行了错误标记（它可能会报告这些库泄漏内存，但实际上没有）。

快速上手：

1. 链接时加上参数 `-ltcmalloc`

   ```
   gcc [...] -o xxx -ltcmalloc
   ```

2. 设置 `HEAPCHECK` 环境变量：

   ```
   HEAPCHECK=normal <path/to/binary> [binary args]
   ```

heap-checker 只在 Linux 上可用。

2.4 cpu-profiler

快速上手：

1. 链接时加上参数 `-lprofiler`

   ```
   gcc [...] -o xxx -lprofiler
   ```

2. 设置 `CPUPROFILE` 环境变量：

   ```
   CPUPROFILE=/tmp/cpuprof <path/to/binary> [binary args] # 指定性能报告文件名
   ```

3. 使用 pprof 来分析 CPU 的使用率

   ```
   pprof <path/to/binary> /tmp/cpuprof      # -pg-like text output
   pprof --gv <path/to/binary> /tmp/cpuprof # really cool graphical output
   ```

CPU profiler 在基于 UNIX 的系统上都可以用，目前在 windows 上不可用。

2.5 动态链接

LD_PRELOAD是Linux系统的一个环境变量，它可以影响程序的运行时的链接（Runtime linker），它允许你定义在程序运行前优先加载的动态链接库。这个功能主要就是用来有选择性的载入不同动态链接库中的相同函数。通过这个环境变量，我们可以在主程序和其动态链接库的中间加载别的动态链接库，甚至覆盖正常的函数库。一方面，我们可以以此功能来使用自己的或是更好的函数（无需别人的源码），而另一方面，我们也可以以向别人的程序注入程序，从而达到特定的目的。俗称偷梁换柱。

当二进制文件不是我们编译的，但又想用这个性能分析工具，就可以通过设置下面的环境变量，当程序执行 `malloc` 和 `new` 时，就会跑去执行 `tcmalloc` 的东西，从而实现我们的目的。

```
# tcmalloc
env LD_PRELOAD=/usr/local/lib/libtcmalloc.so <path/to/binary>
# cpu-profiler
env LD_PRELOAD=/usr/local/lib/libprofiler.so <path/to/binary>
# OR
export LD_PRELOAD="/usr/local/lib/libtcmalloc.so"  # 设置要优先替换的动态链接库
export LD_LIBRARY_PATH=<path/to/library>           # 如果上一个命令找不替换库,用这个试试设置系
unset LD_PRELOAD                                   # 替换结束后解除
ldd <path/to/binary>                               # 查询依赖关系
```

2.6 pprof

执行 `pprof --help` 可查看所支持的输出文件格式。

```
go tool pprof cpuprofile # 分析输出文件
```

3 应用

使用它的过程分为三个部分：将库链接到应用程序，运行代码以及分析输出。

3.1 heap-profiler

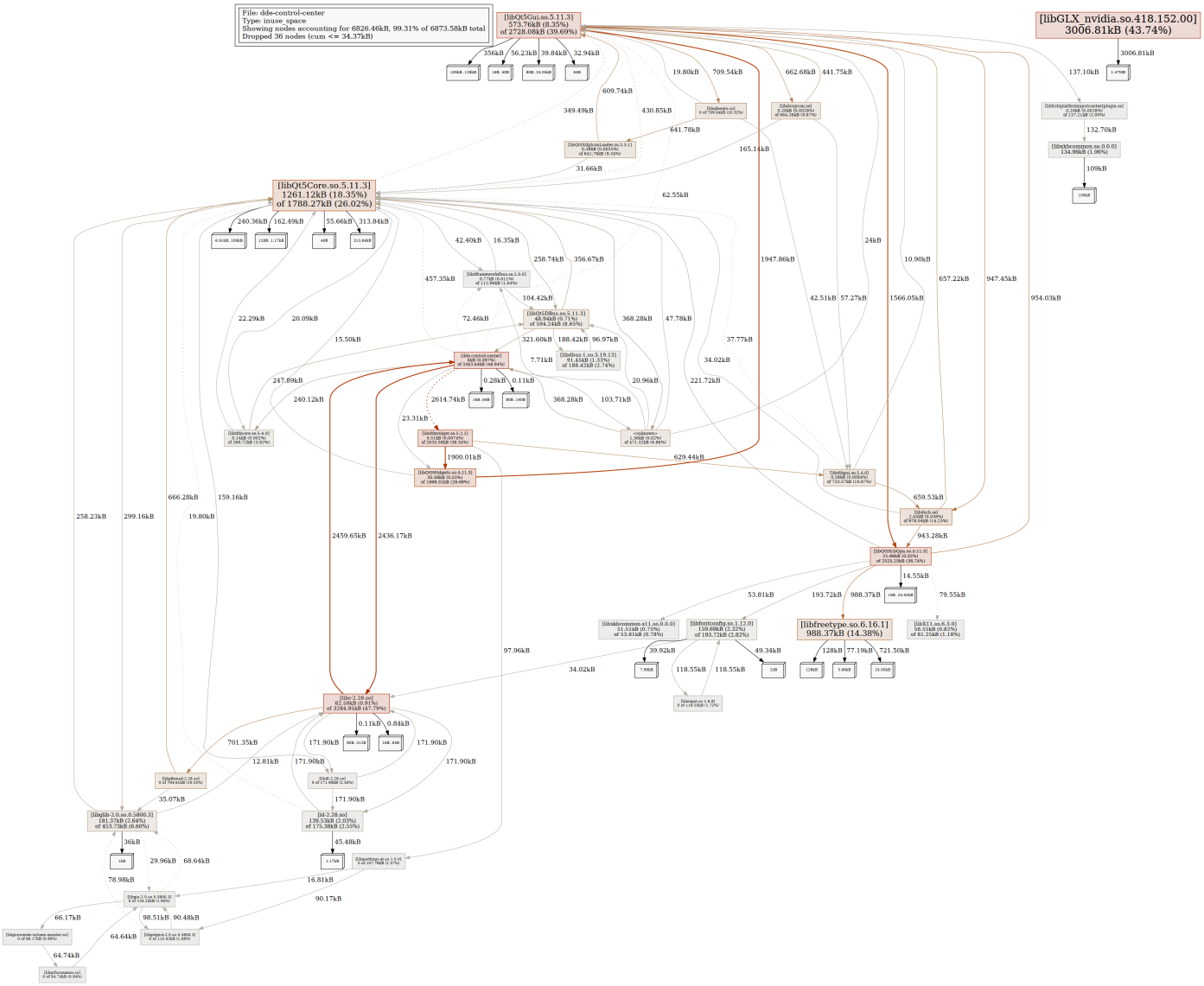
堆内存检测：

1. 在 CMakeLists.txt 设置 `set(CMAKE_CXX_FLAGS "-ltcmalloc -g -Wall -Wl,--as-need -fPIE")`
2. 修改代码

```
#include <gperftools/heap-profiler.h>
int main()
{
   HeapProfilerStart("/home/uos/Desktop/heapprofile"); // 指定输出文件的路径
   // code
   HeapProfilerStop();
   return 0;
}
```

3. 分析数据

```
go tool pprof <path/to/profile>
```



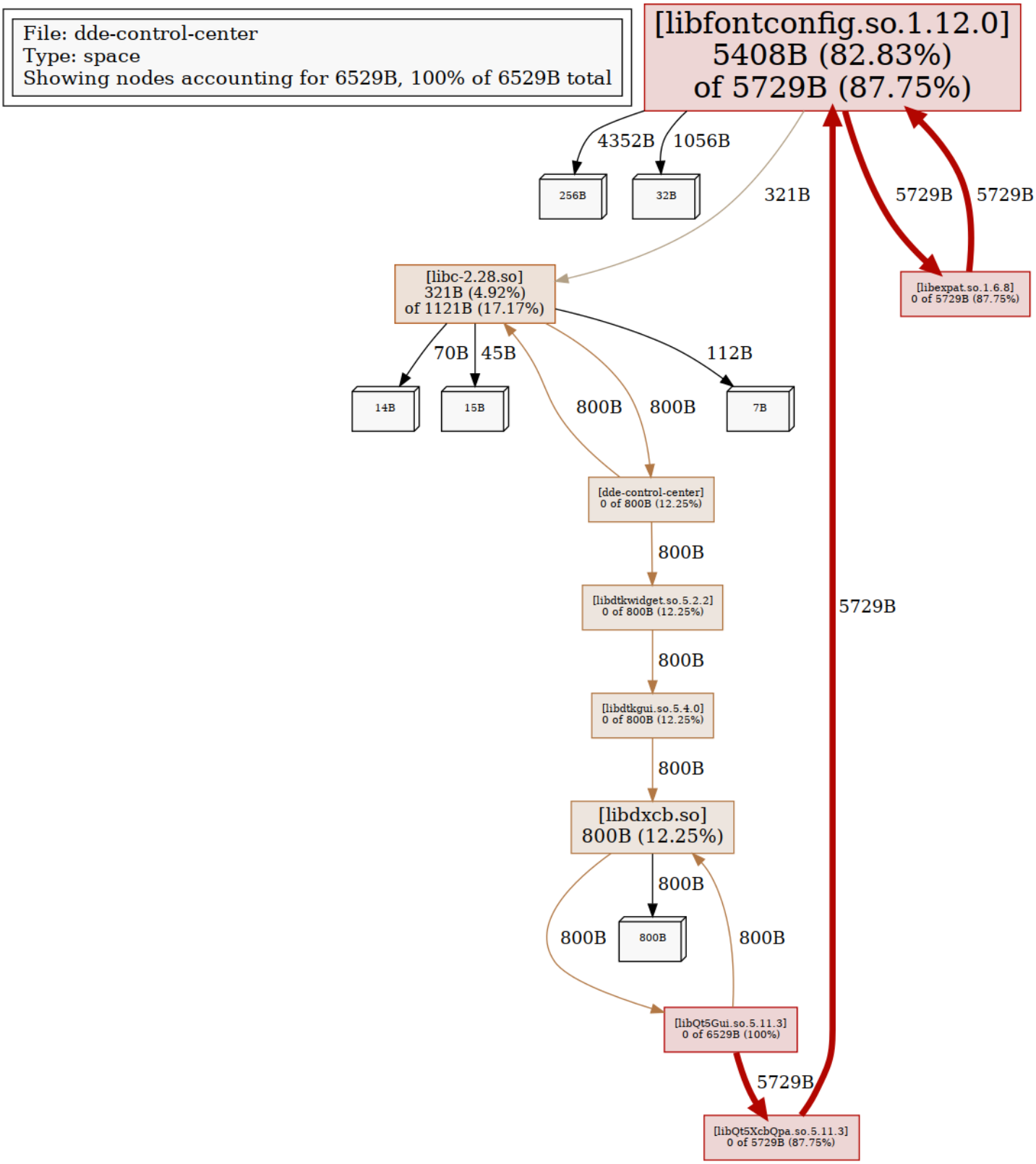
3.2 heap-checker

堆内存泄漏分析：

```
HEAPCHECK=normal <path/to/binary>
```

在执行后的结果中会打印内存泄漏信息，通过执行最后打印的命令以图形化显示结果：

```
google-pprof ./dde-control-center "/tmp/dde-control-center.30298._main_-end.heap" --inus
# or
go tool pprof ./dde-control-center "/tmp/dde-control-center.30298._main_-end.heap"
```



3.3 cpu-profiler

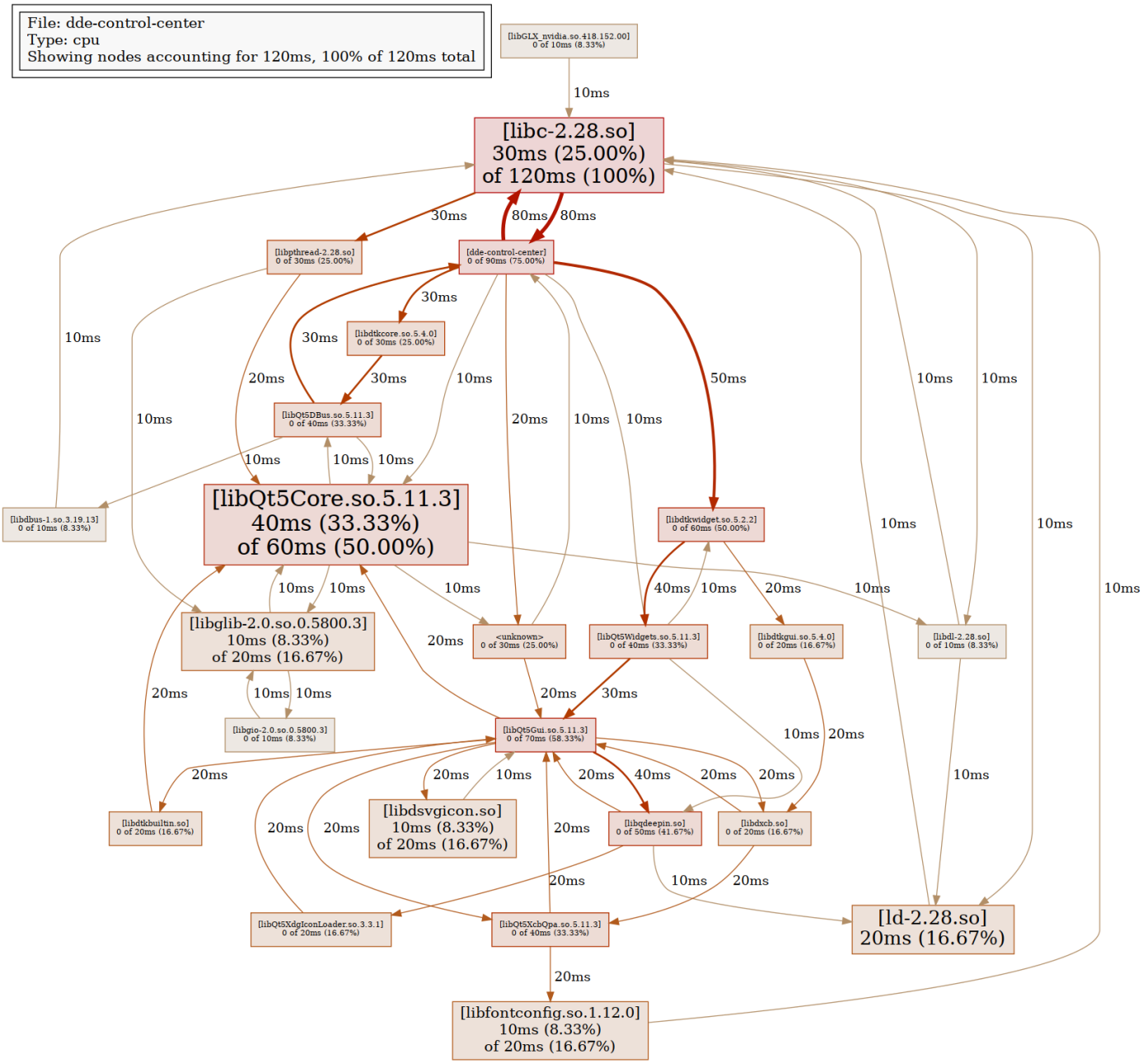
CPU性能监测：

1. 在 CMakeLists.txt 设置 `set(CMAKE_CXX_FLAGS "-lprofiler -g -Wall -Wl,--as-need -fPIE")`
2. 修改代码

```
#include <gperftools/profiler.h>
int main()
{
   ProfilerStart("/home/uos/Desktop/cpuprofile"); // 指定输出文件的路径
   // code
   ProfilerStop();
   return 0;
}
```

3. 分析数据

```
go tool pprof <path/to/profile>
```



4 参考文献

- gperftools 官方文档
- gperftools 官方文档 github pages
- gperftools 官方文档 github wiki
- Google Performance Tools安装以及使用